

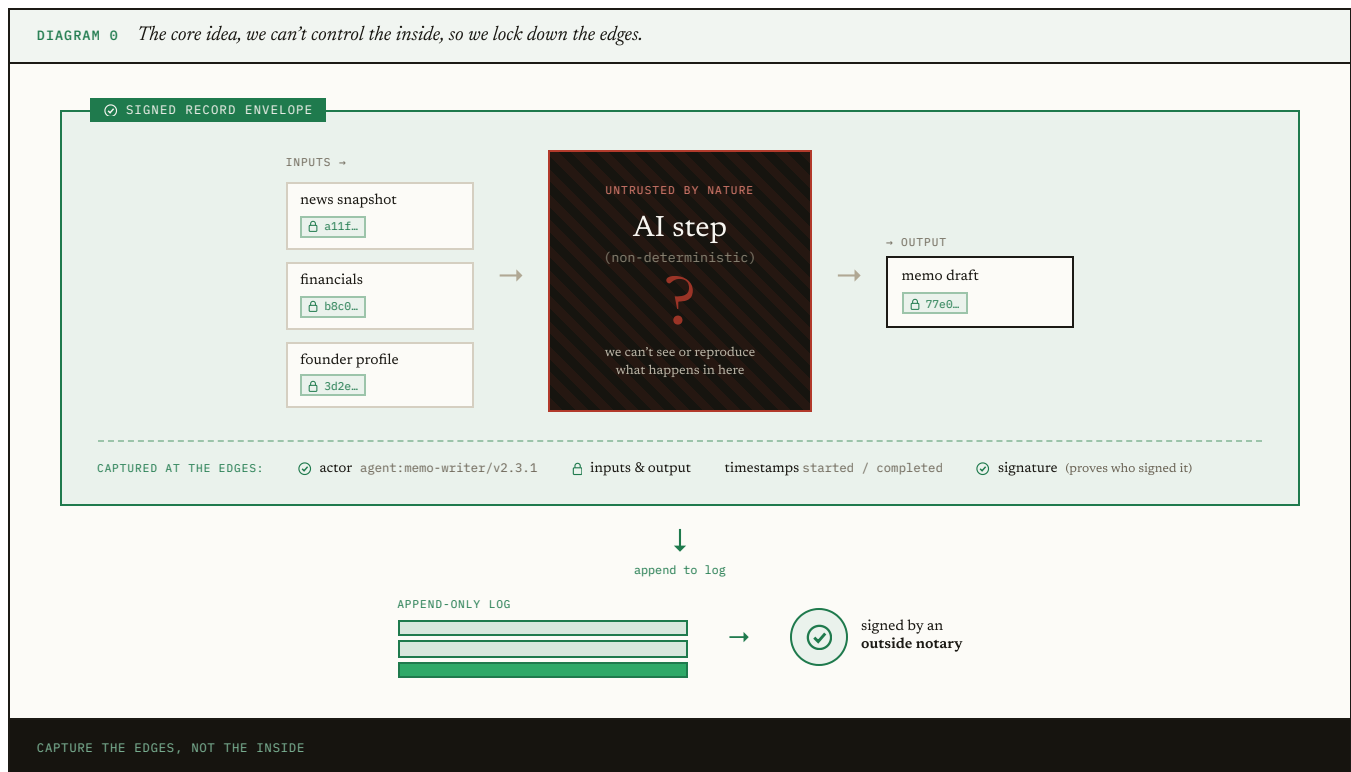
Dominic Lim *Soma Capital case study*

Approach

The design rests on one decision. You cannot make an LLM deterministic, so the goal is to make the *record* of what it did deterministic instead. The reason is temperature: unless you pin a model to a temperature of zero, it will never return exactly the same output twice, so a step can never be retraced by rerunning it. The model stays as unpredictable as it needs to be inside a step. As soon as anything crosses the edge of that step, meaning the inputs it read, the output it produced, who ran it, and when, the system captures it exactly, signs it, and appends it to a log that cannot be rewritten. The unpredictable work stays sealed inside the step, and everything you can observe from outside is exact, attributed, and permanent.

An ordinary database is append-only as a matter of policy, so anyone with admin access can edit a row and cover their tracks. This design is append-only as a matter of math. Records are chained together by hash, so any change to a record at any point in the chain cascades up to a single fingerprint at the top, which reveals the tampering wherever it happened. That top fingerprint is signed by an outside party on a schedule, so checking it needs no trust in Soma at all.

That single move shapes everything else. The brief's two-year projection of 50,000 actions a day works out to roughly one write per second, which sits three to six orders of magnitude below what ordinary databases and disks handle, so raw throughput is never the hard part and the focus belongs on correctness and auditability. The hard part is trust. The guarantee has to hold up against a compromised server, a dishonest insider, and an auditor who has no reason to believe anything Soma says. The whole architecture is built around surviving those three.



1 High-Level Architecture

Requirements

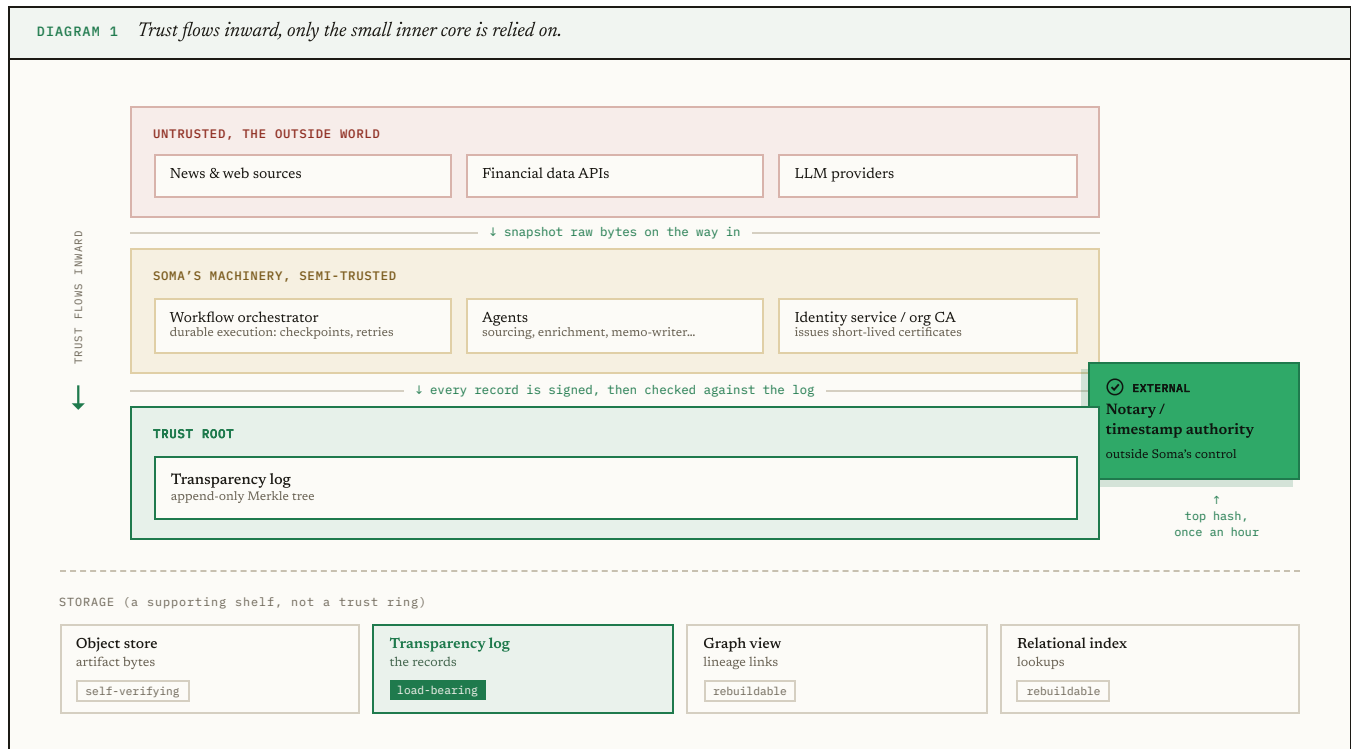
The system has to prove whether information has been modified, by whom, what was modified, and when and where it was made. Each component below follows from one of those requirements rather than being chosen for its own sake.

The question "has this been modified" means an artifact cannot be identified by a name someone could later point at different bytes; it has to be identified by its own contents. That gives a **content-addressed store**, where an artifact's address is a fingerprint of its content, so any change produces a different address. "Who made this, from what, and when" means every step needs a small signed **record** of its actor, inputs, and output. For those records to be tamper-evident too, they cannot live in a database an administrator can quietly edit; they live in an append-only **transparency log**, and because even that log's operator could try to rewrite it, an **external anchor** pins its state somewhere Soma cannot reach.

The rest follow the same way. Actors need identities and keys (the **identity service**), agents fail and retry across long runs without corrupting the record (a durable **workflow orchestrator**), and answering an auditor means walking records back from a decision into something they can check (the **audit layer**). The deeper reason for deriving them this way is that it lets everything hang off a single trusted source: rather than build separate credibility for each of the five provenance answers, the design roots them all in the same anchored log, so the five come out as corollaries of one thing an outsider can check.

Trust boundaries

The central question in a design like this is what you are allowed to trust, because a guarantee is only ever as strong as the weakest thing it depends on. The answer is drawn as three rings.



The outer ring is everything Soma does not control: news sites, financial APIs, LLM providers. It can be wrong, slow, or hostile, so nothing it says is taken as true; the system snapshots exactly what comes back and treats that as a record of what was received, not a fact about the world. The middle ring is Soma's own machinery, the orchestrator, agents, and identity service. It does the real work and holds signing keys, and the point is to avoid trusting it: assume any single machine could be compromised, and require that a lie told here is caught by the inner ring. The inner ring is the trust root, the append-only log and the external notary, kept deliberately small because it is the one thing everyone relies on and a small root is one you can reason about end to end. Trust flows inward. Keeping the root this small is also what makes it sellable: for an LP the whole pitch collapses to one anchored number they can check themselves, and on an already hard sell you want one strong hook rather than many separate claims to defend.

Following one memo through the system

The clearest way to see the pieces work together is to follow a single memo from raw sources to an approved decision, and then watch it get audited. This is the exact path the prototype runs.

A sourcing agent fetches a news article and a financials response and saves the raw bytes of each. Those two saves are the first records, and having no inputs makes them the frontier, the edge where outside data entered. An enrichment agent reads both and writes a founder profile; its first attempt times out, and that failure is written down as its own record, with the successful retry pointing back at it. A memo agent drafts from the profile, a partner edits the draft (creating a new document rather than overwriting the old

one, so the earlier version survives), and the approval pins the exact bytes approved. When the LP later asks the headline question, the system starts at that approval and walks the input links back to the two frontier snapshots: artifacts from the content store, each step a signed record, the whole chain rebuilt on demand.

Durable workflows resistant to failure

There is no need to rebuild the mature infrastructure for reliable execution. A durable engine such as Temporal already checkpoints a workflow's progress so a crash resumes where it left off, and uses idempotency keys so a retried write happens once. The part that matters here is retries, which do two things at once: they raise reliability even when a step is non-deterministic, and they provide a natural checkpoint at which to capture provenance.

The two kinds of step are handled in opposite ways. A deterministic step, such as a database write or an API call, can safely run again, because a second run has the same effect and the idempotency key absorbs the duplicate. A non-deterministic step, such as an LLM call, cannot be run again to check it, because a second run gives a different answer. So the system records the output of a non-deterministic step once and reuses that record, and never re-executes it to verify it. The consequence for provenance is the decision that matters most. Every attempt, including the one that failed, becomes its own permanent record, and the successful retry links back to the failure. A history that drops that failed attempt fails verification, so the boundary is enforced by the math itself instead of by a written policy.

Identity and the trust model

A signature only means something if you know whose key produced it and can be sure it was not quietly stolen months ago. The common approach, long-lived keys plus a revocation list, is where these systems tend to fail: a stolen key stays valid until a human notices and acts. So an organization CA issues short-lived certificates, good for hours, to each agent instance. Rotation then takes care of itself, since an agent just requests a fresh certificate near expiry and there is no long-lived secret to steal. The cost, honestly, is an always-on issuer: if it is down, agents cannot sign. At the rate agents work, keeping an issuer up is far easier than keeping a revocation list correct and fast; for an identity that had to sign offline, long-lived keys would win. There is a second reason I favor this beyond security: validation robust enough to make keys ephemeral is itself a strong point in front of LPs, because it shows the trust story holds without anyone babysitting a revocation list.

Two more pieces complete the model. An agent never acts on its own authority. Its certificate has to carry a delegation chain that ends at a real human, or verification rejects it, so an accountable person always sits at the root of any automated chain. Holding a valid key is also not enough to sign as someone else, because the identity claimed in a record has to match the certificate the CA actually issued. Humans authenticate through Soma's existing single sign-on with hardware-backed keys. The whole trust model fits in one sentence: trust the organization CA for who is who, trust an outside notary for when history existed, and trust nothing else, including Soma's own servers.

Storage, and the five provenance questions

There are four storage layers, and only one has to be trusted. Artifact bytes sit in ordinary object storage, self-verifying because the address is the hash. The records sit in the append-only log, which with the external anchor is the one load-bearing layer. A graph view speeds the audit walk and a relational index answers everyday lookups, and both are derived: an auditor trusting none of Soma's databases could throw them away, rebuild from the log, and every proof would still hold. That leaves the brief's first requirement, five questions each artifact must answer: which agent or human created it, what inputs went in, which version produced it, when it happened, and whether it has changed since. Each is answered by a field in the record, built up next.

2 Deep Dive: the record, the DAG, and the log

This section goes deep on the provenance record, which is where "who made this, from what, and whether it has changed" is actually answered, and where a design that looks fine can be quietly wrong. That is also why it leads the write-up. The rest of the system matters, but the notary and the log are where everything is actually verified, and where it all falls apart otherwise, since without them you could rebuild the Merkle tree and claim it looked that way the whole time. Three things do the work together: the record that captures one step, the hash-links that turn records into lineage, and the log that makes the whole history complete and impossible to rewrite.

What a record captures, and why

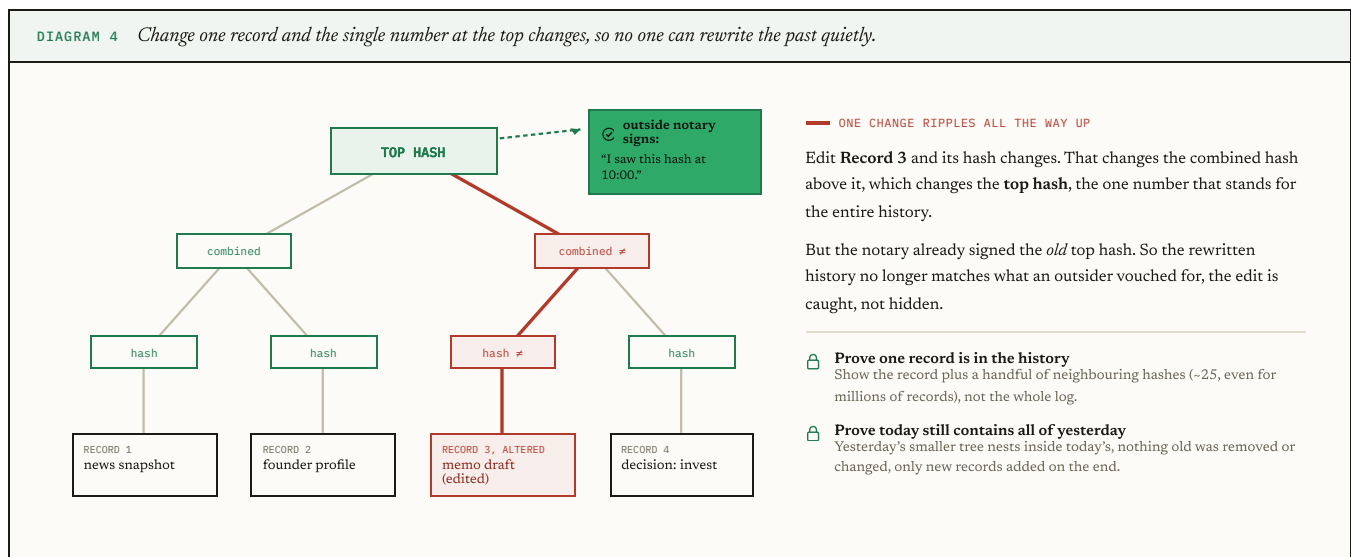
Start from the five questions and let them decide the contents, so the record holds everything it needs and nothing it does not. "Who made this" calls for an actor identity and a signature over the whole record. "From what inputs" calls for a list of the exact upstream artifacts, each named by its content fingerprint. "Which version" puts the agent's version into its name, and for an AI step also the model and its settings. "When" calls for a start and completion time. And "has it been modified" comes for free from naming the output by fingerprint, because any change to the output changes its fingerprint and breaks every record that pointed at it.

Two choices in the record do a lot of work. It names inputs and outputs by fingerprint instead of storing the bytes, so a record stays about a kilobyte whether the artifact is a 100-byte CRM update or a 50 KB memo, and so deleting an artifact's bytes for a privacy request never disturbs the records that reference it. And a failed attempt is a full record that the successful retry links back to, which makes hiding a failure structurally impossible instead of a matter of good behavior. One field exists purely for the AI problem. The system cannot record why a model produced a given output, because that lives inside a black box no one controls. What it can record is everything that went into the call: the model and version, the exact prompt, the settings, and the provider's own request ID. The step becomes fully attributable even though it can never be reproduced. That gap between attributable and reproducible is the honest center of putting AI into an audit trail.

Hash-links are the lineage, and the log makes it complete

Each record names its inputs by fingerprint, and each of those inputs was the output of an earlier record that named its own inputs the same way, back to the frontier. The records therefore form a graph whose edges are fingerprints sealed inside signed records. Faking an ancestor would require either two different documents with the same fingerprint, which the hash function is built to make infeasible, or a freshly forged signed record, which cannot be slipped into the log after the fact. Lineage and tamper-resistance become the same mechanism, so there are no two systems to keep in agreement.

A signature only stops a record from being changed. It does nothing to stop a record from being deleted, and nothing to stop someone from keeping a second, fully signed, invented history on the side. So the records go into a structure where no one, not even the operator, can delete one or swap in a fake without it showing. Hash every record, hash those hashes in pairs, and keep going until a single hash sits at the top. That top hash is a fingerprint of the entire history at once, and changing any record anywhere below it changes the top hash. This structure is a Merkle tree, the same construction that keeps the world's website certificates honest under Certificate Transparency, which is good evidence that it holds up against real adversaries at scale.



Two useful proofs come out of the tree almost for free, and they happen to be exactly what an auditor needs. You can prove that one record is in the history by handing over just that record and about 25 neighboring hashes, instead of the whole log. And you can prove that today's history still contains all of yesterday's, unchanged, with new records only added on the end. One gap remains, which is that all of this assumes someone actually kept yesterday's top hash. If Soma held everything, it could rewrite the past and then lie about what the old top hash was. So the top hash does not live only inside Soma. Once an hour, the current top hash goes to an outside timestamping notary whose only job is to sign "I saw this hash at this time," the same neutral service used to timestamp legal documents. From that moment the top hash is pinned where Soma cannot reach it, and any later attempt to rewrite even one old record fails to produce a valid proof back to the hash the notary signed. Rewriting history stops being something forbidden by policy and becomes something caught by arithmetic.

Answering the LP with a proof that needs no trust in Soma

The audit layer takes the approval artifact, walks the input links back to the frontier, pulls in the failed attempts along the way, and packages the result as a proof bundle. The bundle holds every record with its inclusion proof, the current and last-anchored top hashes with a consistency proof between them, every certificate involved, the notary's receipt, and, if wanted, the document bytes. What makes this a proof rather than a report is what an outsider can do with it. A standalone verifier checks the entire bundle while trusting exactly two public keys, the organization CA and the notary, and it shares no database or code path with the system that produced the bundle. If the bundle is malformed in any way, the verifier returns a failure instead of crashing, so a crash can never be mistaken for a pass. The same verifier defeats the four ways someone would actually try to cheat: editing an approved memo after the fact breaks its fingerprint; forging a record under a lookalike CA fails to chain to the real CA; hiding a failed attempt leaves the retry pointing at a record that is missing; and rebuilding the whole log with one record swapped cannot produce a consistency proof back to the notary's hash.

Two places this kind of system goes quietly wrong

Two subtle failures are worth calling out, because both are failures of attribution, the exact property the system exists to guarantee, and both stay invisible until someone deliberately lies. The first is timestamp handling. In JavaScript every comparison against an invalid date returns false, so a broken timestamp reads as being inside a certificate's validity window, and an expired certificate slips through. The guard is to reject unparseable times and fail closed. The second is the difference between signing an identity and binding it. Checking the signature and the certificate is not enough, because the identity claimed in the record also has to match the certificate's subject, or any valid key holder could attribute work to someone else. The guard is an exact match on the claimed identity and delegation chain. The pattern behind both is the same: the dangerous gaps are in the checks a verifier forgets to make, which is why the tests that matter most are the ones that try to lie to it.

3 Tradeoffs and Future Direction

Every choice traces back to one goal: verifiability by a party who does not trust Soma. The rest I gave up on purpose. Throughput was never in question at one write per second. Storage stayed deliberately boring, since boring storage is easier to trust than something clever. And audit queries may take the minute the brief permits, which lets query indexes be rebuilt from the log rather than maintained as fragile always-correct ones.

Three forks are worth naming. A blockchain would remove the trusted log operator entirely, but a permissionless chain needs validators, which exposes the network and has to pay them to run, all out of proportion for an internal tool whose rewrite window is already held to an hour by the notary. It stays an escape hatch: anchoring the top hashes into a public chain is one integration away if that requirement ever arrives. A separate lineage database plus an audit log would be simpler, but it splits lineage and integrity into two systems that can drift, leaving the auditor trusting the join between them; making the

signed record stream itself be the lineage removes that seam. Re-running steps to verify them stays available for deterministic steps, but cannot work for LLMs, since a second run differs and providers do not sign their outputs, so the design records the full call context now and can upgrade to provider-signed inference when it exists.

The limits are worth stating plainly. The system proves lineage, not truth: a hallucination is recorded with perfect integrity, which is why decision-grade artifacts require a human approval, putting an accountable person in every real decision chain. That reflects a deliberate choice, talked through with Alan: lean on heuristics for the bulk of the work and bring LLMs in only at gating points, since a smaller non-deterministic surface means fewer decisions riding on something no one can re-derive. A live, compromised endpoint can sign real lies with a valid key; short certificate windows and the anchored log contain that without removing it, and I scoped it down on purpose because this is an internal tool, where the threat model leans toward honest mistakes and insider edits rather than a hostile public. Outside data is trusted on first fetch and fingerprinted immediately, so a dispute reduces to what their API returned at 14:02 UTC. Record timestamps are self-claimed, with per-record countersigned timestamps from the notary as the ready fix.

On what I would build next: at 10x to 100x nothing in the trust core changes shape. The step I care about most follows the heuristics-first stance, pushing more of each workflow into deterministic, checkable steps and narrowing the LLM toward pure gating, so the share that can only be attributed rather than reproduced keeps shrinking. Beyond that the plumbing scales without touching the guarantee: the log shards per fund or workflow class under a shared super-root, anchoring batches, cold bytes tier to archival while fingerprints stay hot, and the derived views move to a dedicated graph store because they are safe to rebuild. What you could prove to the LP on day one is what you can still prove on the ten-thousandth.